

Functional Safety and Self-Diagnostics of a Light Emitter ECU – OPmobility Challenge 2025

Patricio Gonzalez-Salinas
Escuela de Ingenierías y Ciencias
Instituto Tecnológico y de Estudios Superiores de Monterrey
Monterrey, México
A00838858@tec.mx

Angel Eduardo López-Solís
Escuela de Ingenierías y Ciencias
Instituto Tecnológico y de Estudios Superiores de Monterrey
Monterrey, México
A00837012@tec.mx

Samuel Granados-Barrios
Escuela de Ingenierías y Ciencias
Instituto Tecnológico y de Estudios Superiores de Monterrey
Monterrey, México
A00228152@tec.mx

Juan Antonio Velázquez-Mendoza
Escuela de Ingenierías y Ciencias
Instituto Tecnológico y de Estudios Superiores de Monterrey
Monterrey, México
A01067863@tec.mx

Abstract— This project presents the design and development of a prototype diagnostic system for an Electronic Control Unit (ECU) used in automotive lighting modules. The challenge, proposed by OPmobility, consists of implementing a controller capable of monitoring the operating conditions of an LED assembly using a microcontroller based on AVR architecture (ATmega328P integrated in an Arduino UNO development board). The system uses analog measurements of voltage, current, and temperature, as well as bidirectional UART communication for fault and status reporting. The proposed solution integrates: (1) analog circuits based on voltage dividers, thermistors, and current sources; (2) a processing module using a 10-bit ADC; (3) a UART communication interface capable of sending diagnostic frames and receiving commands; and (4) a fault-counting mechanism for modes such as overvoltage, undervoltage, open LED, short circuit, and overtemperature. The results show that the integrated system performed reliably across all diagnostic scenarios. The prototype accurately detected voltage and temperature anomalies, executed automatic protection actions, adjusted LED brightness dynamically, logged faults persistently in EEPROM, and communicated detailed status information over UART with no transmission errors. These results confirm that the developed diagnostic architecture behaves consistently and fulfills all required system objectives.

Keywords—ECU, OPmobility, LTspice, methodology, Multisim, Simuli

I. INTRODUCTION

A. General context

OPMobility plays a major and increasingly influential role in the automotive industry by offering a wide range of innovative solutions that encompass vehicle performance, safety, and long-term sustainability. The company is strongly committed to the development of advanced components and integrated systems that facilitate the global transition toward electrification and smart mobility. Its portfolio brings together cutting-edge technologies designed to reduce environmental impact, enhance energy efficiency, and meet the growing technical and regulatory demands of the global market. Through ongoing research efforts and strategic collaboration with international automotive leaders, OPMobility actively contributes to shaping the future of mobility while upholding rigorous standards of quality, reliability, and technological excellence.

Within the context of this challenge, the company presents the need for a system capable of diagnosing and monitoring an automotive lighting module, highlighting the importance of accessible tools that can detect electrical and thermal failures during the early stages of

product development. In response to this requirement, the present project proposes the design and implementation of an ECU prototype based on the ATmega328P microcontroller. This prototype is intended to measure critical operating parameters, identify abnormal conditions, and establish bidirectional communication via UART. By demonstrating that a low-cost architecture can successfully replicate essential automotive diagnostic functions, the project positions the system as a versatile prototyping platform for future applications in vehicle lighting and related domains.

B. Target group

The object of study focuses exclusively on the construction and validation of an ECU prototype based on an ATmega328P microcontroller (Arduino UNO) for an automotive lighting module, conceived as an accessible and functional tool for early-stage diagnostics of electronic systems. This development benefits multiple actors within the sector: automotive component manufacturers, by providing a cost-effective testing system to evaluate electrical behavior and validate diagnostics prior to final design; electronic and automotive engineers, by offering a modular prototype that facilitates the study of faults and the review of functional safety strategies; and students and educational institutions, by granting a didactic platform for understanding automotive architecture, signal measurement, embedded diagnostics, and inter-module communication. Collectively, this prototype reduces time and cost in initial testing stages, enables the validation of faults without specialized equipment, facilitates the analysis of analog measurements and bidirectional communication, and contributes to a deeper understanding of intelligent lighting systems and the functional safety principles that govern them.

C. Problem statement

Automotive lighting systems require reliable diagnostics to prevent failures that could compromise vehicle safety; however, current commercial solutions rely on complex ECUs integrated into automotive networks, whose cost and level of specialization hinder their use during early prototyping stages. In this context, there emerges a need for an accessible tool capable of detecting failures in LED modules, logging events, applying basic functional safety measures, evaluating thermal and electrical behavior, and validating communication between the ECU and the

lighting module. The central problem lies in the absence of an affordable and modular prototype that allows students, engineers, and manufacturers to simulate, measure, and diagnose typical failures in automotive lighting systems without depending on advanced ECUs or specialized equipment. For this reason, the development of a controller based on low-cost microcontrollers is proposed, capable of replicating essential diagnostic functions and serving as a validation platform in educational and engineering environments.

D. Justification

The present project is justified by the need for an accessible and functional system for early-stage diagnosis of failures in automotive lighting modules, employing a platform capable of validating electrical and thermal parameters, logging events, and applying basic functional safety measures during initial design phases. Although the PIC18F45K50 microcontroller was initially considered, an irreparable configuration fault and the lack of available units in stock required a migration toward an alternative that would ensure development continuity without compromising the technical scope of the system. In this context, the Arduino UNO—based on the ATmega328P—represents an appropriate solution for automotive prototyping due to its rapid integration, immediate availability, higher processing efficiency, and stability during experimental testing. These elements make it possible to move the project forward and produce a working prototype with available resources.

In addition to these practical advantages, the ATmega328P provides the technical capabilities required by the project: a 10-bit ADC for voltage and temperature measurement, full-duplex UART communication for ECU–LED module exchange, and an architecture sufficiently robust to implement fault detection logic and event counting. With this combination of attributes, the proposed system directly addresses the identified needs: it enables the detection of electrical and thermal faults through analog sensing, provides clear logging of operational events, validates communication with the lighting module without the need for specialized equipment, and applies responses to abnormal conditions such as brightness reduction or safe shutdown. Likewise,

the use of Arduino facilitates experimentation and design adaptation, offering an economical and modular platform that reduces cost compared to full automotive ECUs—making it particularly useful for students, engineers, and manufacturers seeking to validate designs in controlled environments before scaling to production-grade systems.

E. Theoretical Framework

Microcontrollers: A microcontroller is an integrated computational system that combines a CPU, memory, timers, digital I/O ports, an ADC, and communication modules. The ATmega328P, used in this project, implements a Harvard architecture, operates at 16 MHz, includes a 10-bit ADC, and provides hardware UART, making it well suited for automotive prototyping.

Harvard Architecture in the ATmega328P: The Harvard architecture separates program memory from data memory, enabling simultaneous access and stable execution times. These benefits embedded automotive applications where predictability is essential.

Microcontroller Application in Prototyping: Microcontrollers allow the validation of control algorithms, diagnostic systems, and fault-handling strategies without relying on commercial ECUs. Their low cost and adaptability make them key tools in early design phases.

Communication Protocols Used: The system employs full-duplex UART, an asynchronous 8N1 protocol that enables bidirectional communication between the prototype ECU and the LED module. UART offers simplicity, flexibility, and adequate speed for transferring commands and diagnostic codes in a prototyping environment.

Literature Review: Specialized literature presents various methodologies for detecting faults in automotive LED modules. Seritan et al. describe a system based on Vsense measurement that allows the identification of open-circuit and short-circuit conditions through voltage analysis on the load. Other works employ thermal sensors, current measurements, or combined PWM/feedback strategies. These investigations demonstrate that analog acquisition and embedded analysis are feasible tools for early-stage diagnostics.

Similarly, studies such as Attal (2025) highlight the use of low-cost microcontrollers as tools for tracing and analyzing anomalous behaviors in ECUs, showing that prototyping platforms can facilitate fault validation before implementing final automotive architectures.

F. Objectives

The objectives of the project are formulated based on the functional and design requirements established for the ECU prototype, which define the minimum capabilities the system must possess to properly evaluate the behavior of an automotive lighting module. Among these requirements, the detection of common faults in LED assemblies—such as open circuit, short circuit, overvoltage, undervoltage, and overtemperature—stands out, supporting the need to implement an analog sensing system using the ATmega328P's ADC and the integration of thermal resistor NTC for thermal monitoring. Likewise, the importance of maintaining reliable communication between the ECU and the LED board establishes the need to develop a full-duplex UART interface capable of managing data exchange and validating the module's operational state in real time.

Another essential requirement of the prototype is the ability to record the number of faults detected during operation, which is reflected in the objective of designing a persistent counting mechanism to analyze the nature and frequency of registered events. Similarly, requirements associated with basic functional safety principles—such as maintaining system operation in the presence of a single fault, adjusting brightness under critical thermal conditions, or activating a safe state when multiple faults are detected—guide the development of the controller's response logic. Finally, hardware constraints such as tolerance to input signals up to 12 V in the sensing stage, operation at 5 V/20 mA per LED channel, and the need to maintain independent power sources per channel establish the technical framework in which the specific objectives are articulated. Together, these elements clearly define the scope of the project and ensure coherence between system requirements and the operational goals of the prototype.

6.1.1

Implement an analog measurement system using the ATmega328P's ADC to monitor voltage, temperature, and diagnostic signals of the LED module.

6.1.2

Detect the main electrical faults of the lighting system—open circuit, short circuit, overvoltage, and undervoltage—through sensing circuits and threshold evaluation.

6.1.3

Integrate an thermistor sensor to identify overtemperature conditions and trigger the corresponding system responses.

6.1.4

Develop a full-duplex UART communication interface to enable secure data exchange between the prototype ECU and the LED board.

6.1.5

Design a fault-counting mechanism that records incidents occurring from the beginning of system operation.

6.1.6

Apply basic functional safety strategies, such as maintaining operation under a single fault, adjusting LED brightness under overtemperature conditions, and activating a safe state when two or more simultaneous faults are detected.

6.1.7

Simulate controlled faults using potentiometers, BJT transistors, switches, and voltage variations to validate system behavior under different scenarios.

6.1.8

Experimentally evaluate the performance of the prototype, verifying fault detection, UART communication stability, and the effectiveness of the implemented safety mechanisms.

G. Hypothesis

The project hypothesis is based on the assumption that a simple, low-cost microcontroller architecture, such as the ATmega328P integrated into the Arduino UNO, is capable of effectively replicating the essential diagnostic functions used in commercial automotive systems during the prototyping stage. The premise is that the electrical and thermal phenomena characteristic of a lighting module—voltage variations, interruptions in the LED load, short circuits, and temperature increases—can be accurately detected through analog measurements and

basic sensors, provided that the processing logic and decision thresholds are correctly defined. It is also assumed that full-duplex UART communication provides the level of reliability required to manage information exchange between the prototype ECU and the LED board, allowing operational states to be validated without the need to employ more complex automotive protocols.

Under these assumptions, the testing system developed is expected to consistently identify electrical and thermal faults, log relevant events from the beginning of operation, and apply responses based on elementary functional safety principles, such as maintaining operation under isolated faults or activating a safe state when multiple faults occur. Likewise, it is anticipated that the integration of simple circuits—voltage dividers, thermal resistor, and BJT transistors—will be sufficient to evaluate system behavior without relying on specialized hardware. Overall, the prototype is expected to fulfill its purpose as an economical and modular platform for validating diagnostic and control concepts prior to migrating to a more complex automotive ECU.

II. PROPOSAL

A. Methodology

1) Process Acquisition and normalization of analog signals

The first process focused on enabling the microcontroller to measure the electrical variables of the lighting module. The ATmega328P ADC was configured using AVCC as the reference voltage and enabling channel ADC5, corresponding to the thermistor sensor used for thermal monitoring. In parallel, a voltage divider was designed and constructed to sense the electrical behavior of the LED load. Once the analog interfaces were established, conversion tables between ADC readings and real voltages were generated, allowing the definition of thresholds for overvoltage, undervoltage, and open-load detection.

2) Process Implementation of UART communication

The second process aimed to establish a reliable communication link between the prototype ECU and the LED module via UART. Registers UBRR0H and UBRR0L were configured to operate at 115200 bps, and the transmitter and receiver were enabled through bits TXEN0 and RXEN0. A one-byte command structure was implemented to encode the instruction type, the target channel, and the required action. A key part of this stage was validating the “alive check” mechanism, which involved sending command 0xFE and expecting 0xFF in response as a connectivity confirmation. Serial communication was validated using echo tests and frame integrity analysis.

3) Process Integration of electrical and thermal fault detection

With measurement and communication already established, the next step involved developing the diagnostic logic that interprets the acquired values. An algorithm was implemented to compare ADC readings against defined thresholds and classify each fault mode: overvoltage, undervoltage, overtemperature, open-load and short-circuit. Thermistor temperature readings were compensated according to the 10 K Ω at 25°C relationship. Fault counters were also implemented, and indicator LEDs were programmed to illuminate under critical conditions, allowing visual verification of system behavior.

4) Process Unit testing of peripherals

Before integrating all modules, each peripheral was subjected to unit testing to ensure proper independent operation. The ADC was evaluated by transmitting digitized readings through UART and verifying consistency between measured values and test conditions. UART communication was validated by transmitting test patterns such as 0xAA and 0xFF to ensure link stability. GPIO pins were also tested by confirming LED activation and switch detection. Each test was documented using standardized test tables and photographic evidence.

5) Process Full system integration

Once each peripheral performed correctly, progressive system-level integration was carried out. This phase combined analog sensing, diagnostic logic and serial transmission within the firmware’s main loop.

Synchronization between measurement and frame transmission was calibrated, and system behavior was evaluated while interacting with the LED load under both normal and simulated fault conditions. Each integration iteration allowed refining the implementation and improving the overall operational flow.

6) Process Final validation and iterative refinement

During the final stage, the prototype underwent complete validation, including response time analysis, fault detection accuracy and UART stability under varying conditions. Thresholds used for analog measurement were adjusted after observing electrical noise in real conditions, and all functions were retested until reliable and repeatable detection was achieved. This process concluded with the full documentation of system behavior and verification that the prototype met the requirements initially defined by the partner organization.

B. System Implementation

1) Peripheral configuration: The system was implemented using an ATmega328P microcontroller running at 16 MHz, integrated on an Arduino Uno-type platform and programmed in C through direct register manipulation. The firmware configures and coordinates several internal peripherals—GPIO, timers, the ADC, UART and EEPROM—to enable the diagnostic functions of the LED channels, keypad interaction, temperature and supply-voltage monitoring, as well as persistent fault storage.

The first stage involved configuring the digital input and output pins. Port B was assigned for controlling the four LED channels through pins PB2, PB3, PB4 and PB5, which are set as outputs within the `init_pins()` routine and initially driven high to enable current delivery to each channel. The same port, together with port D, is used for interfacing the 4×4 matrix keypad: the row lines are connected to PB1, PB0, PD7 and PD6, configured as inputs with internal pull-up resistors enabled, while the column lines are connected to PD5, PD4, PD3 and PD2, which are configured as outputs initially driven high. The function `get_keypad_key()` performs the column-by-column scanning by pulling one column low at a time and reading the state of the row inputs to identify the pressed key, using short delays through `delay()` to mitigate the effects of mechanical bouncing.

Timer/Counter0 is used as the primary time base for LED control and ADC synchronization. In the `timer0_init()` function, the timer is configured in Fast PWM mode via the `TCCR0A` register, and a prescaler of 1024 is selected in `TCCR0B` to achieve an interrupt rate suitable for LED switching. Additionally, both compare match (`COMP_A`) and overflow interrupts are enabled through `TIMSK0`. During the overflow interrupt service routine (`TIMER0_OVF_vect`), and provided that no error condition is active, the firmware drives PB2 through PB5 high to turn on the enabled LED channels and simultaneously triggers a new ADC conversion by setting the `ADSC` bit in `ADCSRA`. Later, in the compare-match interrupt (`TIMER0_COMP_A_vect`), these pins are driven low, defining the effective duty cycle of the software-generated PWM applied to the LED loads.

To support these operations, Timer/Counter2 is used to generate short delays primarily for keypad scanning. The `delay()` function temporarily configures Timer2 in Fast PWM mode, selects a prescaler of 1024 and loads `OCR2A` with 255 to produce a full counting period. The firmware waits for the corresponding overflow condition by polling the `TIFR2` flag, after which the timer is stopped, and the flag cleared. This mechanism provides a simple and sufficiently stable time reference for implementing keypad debouncing and small inter-column delays during matrix scanning.

The ATmega328P's analog-to-digital converter is responsible for periodically sampling six different signals related to system status. Initialization is performed in `adc_init()`, where `AVcc` is selected as the voltage reference via `ADMUX` and the ADC prescaler is set to 128 in `ADCSRA`, ensuring a proper conversion clock given the 16 MHz system frequency. The ADC is enabled together with its interrupt, so that each completed conversion triggers the `ADC_vect` routine. In this ISR, the 10-bit conversion result is stored in the `adcs[]` array at the index indicated by `adc_select`. The function `adc_next_channel()` implements a round-robin scan across six channels, updating `ADMUX` to select the next channel and asserting the `adcs_completed_flag` once a full sampling cycle has been completed. Functionally, `adcs[0]` through `adcs[3]` correspond to the measurements of the four LED channels, `adcs[4]` stores the temperature value provided by the thermistor sensor and `adcs[5]` monitors the supply voltage (`VCC`). These readings are later evaluated by the `handle_fault_logic()` routine, which compares them

against predefined thresholds to detect overvoltage, undervoltage, open-LED and short-LED conditions, as well as overtemperature, adjusting the PWM duty cycle and channel state accordingly.

Asynchronous serial communication is implemented through the UART (`USART0`). The `uart_init()` function establishes the baud rate by loading the `UBRR` value into `UBRR0H` and `UBRR0L`, enables double-speed mode via the `U2X0` bit in `UCSR0A` and activates both the transmitter and receiver, as well as the receive interrupt, through `UCSR0B`. The frame format is configured as 8 data bits and 1 stop bit with no parity using `UCSR0C`. Received bytes are captured in a small buffer within the `USART_RX_vect` interrupt service routine, while data transmission is handled through the `uart_transmit()` and `uart_transmit_string()` functions. This interface is used to report channel states, the accumulated error count and diagnostic messages when overvoltage, undervoltage or LED faults are detected.

Finally, the microcontroller's internal EEPROM is used to store the accumulated number of detected errors between power cycles. At startup, within the `main()` function, the variable `error_count` is initialized by reading address zero of the EEPROM through `eeprom_read_byte(0)`. Each time a new fault is detected—whether due to out-of-range supply voltage or LED channel anomalies—the error counter is incremented, and the updated value is written back to EEPROM using `eeprom_write_byte()`. These functions manage the EEPROM control registers directly, ensuring that no write operation is in progress before initiating a new transaction.

Overall, the configuration of the peripherals described above enables the prototype to perform coordinated analog signal acquisition, software-based PWM generation for LED control, user interaction through the matrix keypad and diagnostic communication through UART, while maintaining a basic persistent error log through the EEPROM. The physical implementation of the system is complemented by a hardware assembly integrating the ATmega328P, the 4×4 keypad, the LED module, the thermistor temperature sensor and the VCC sensing network, all of which are documented with photographs in the corresponding infrastructure section.

2) System components:

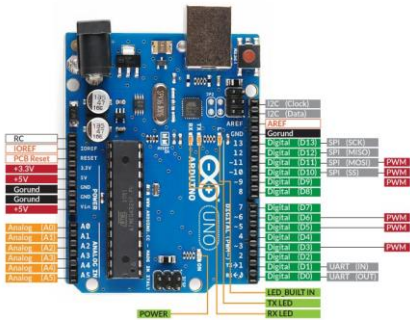


Figure 1. ATmega328P module Arduino UNO.



Figure 2. Potentiometer 1 k Ohm.

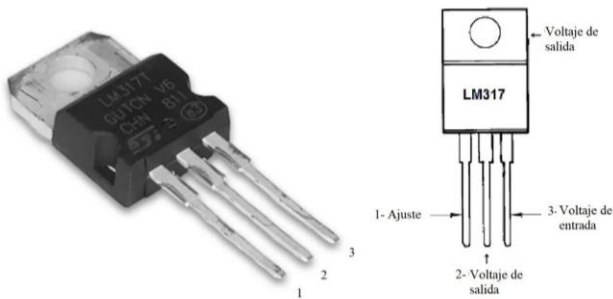


Figure 3. Voltage regulator LM317.



Figure 4. Multiple identical LEDs are used in the system (12 pieces).



Figure 5. Multiple identical BJT 2N3904 are used in the system (4 pieces).



Figure 6. Multiple identical BJT 2N3906 are used in the system (5 pieces).



Figure 7. Thermistor sensor

Color band	Value	Quantity
	10 K	1
	7.5 K	4
	5.1 K	4
	1 K	4

Figure 8. Resistors table.

C. System Testing

1. Unit test

Unit tests are used to validate, in isolation, the correct operation of each fundamental peripheral of the system before integrating them together. This approach ensures that modules such as the UART, ADC, GPIO and timers perform according to their specifications, allowing early detection of faults and minimizing the risk of errors during full system integration. The following section presents the unit tests carried out for the key peripherals used in the diagnostic prototype.

	PU-1/03-DIC-2025			
Team	OPmobility – Diagnóstico LED			
Responsible	All the team			
Description	Verification of the UART (USART0) peripheral functionality through message transmission to the serial monitor.			
Function	Asynchronous serial communication for system status diagnostics.			
Components	ATmega328P (UART0), USB cable, PC with serial monitor.			
Hardware	ATmega328P microcontroller @ 16 MHz, UART configured at 115200 bps, 8N1, 4×4 keypad.			
Procedure		Expected behavior with acceptance criteria	Actual behavior	Result passed or not
	1. Power the system and connect the UART to the serial monitor (115200 bps, 8N1)	When pressing "A", the message "Channel 0 state: X" is printed.	The message "Channel 0 state: X" was displayed correctly when "A" was pressed.	OK
	2. Press the "A" key on the keypad.	When pressing "B", "C", or "D", the corresponding states are printed.	The system printed the corresponding states for keys B, C, and D without delay or errors.	OK
	3. Observe the message transmitted via UART.	When pressing "#", the message "Total errors: N" is displayed.	errors: N" was correctly displayed when "#" was pressed.	OK
	4. Repeat with keys "B", "C", and "D"	No corrupted characters or loss of information should appear.	No corrupted characters, missing data, or transmission errors were observed.	OK
	5. Press "#" to request the total number of logged errors.	Acceptance criteria: All messages must be correctly received on the serial monitor on the first attempt.	All messages were received correctly on the first attempt without retransmission.	OK
Comments	We repeated the test twice because we initially forgot to press channels B, C, and D. For this reason, two test records are shown.			

Figure 9. Unit test 1

A his test verifies the correct transmission of data from the microcontroller to the serial monitor on the host computer. By interacting with the keypad, the system sends messages indicating the state of each LED channel and the accumulated error count. The test confirms that the UART operates at the configured baud rate (115200 bps), with no loss of data or corrupted characters, ensuring the reliability of the communication link used for system diagnostics.

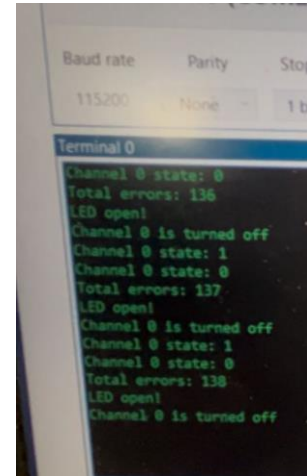


Figure 10. Unit test record 1

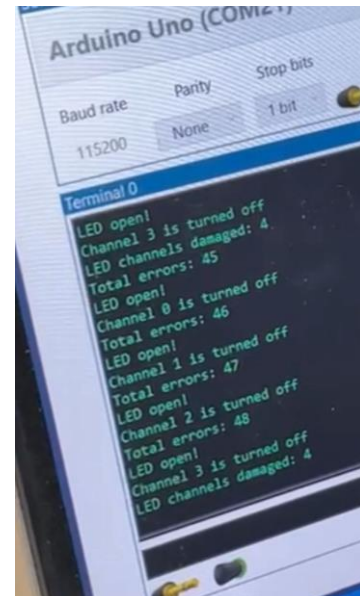


Figure 11. Unit test record 2

2. Unit test

Team	PU-1 /03-DIC-2025			
Responsible	OPmobility – Diagnóstico LED			
Description	Verification of ADC performance using the LM35 temperature sensor and automatic PWM brightness adjustment.			
Function	Analog-to-digital conversion and dynamic LED intensity control based on temperature.			
Components	ATmega328P (ADC, PWM), LM35 sensor, LED channel, USB cable, PC with serial monitor.			
Hardware	ATmega328P @ 16 MHz, ADC 10-bit, PWM output configured for LED control.			
Procedure		Expected behavior with acceptance criteria	Actual behavior	Result passed or not
	1. Power the system and ensure the LM35 sensor is connected.	The ADC initializes correctly and begins sampling the LM35 output voltage.	ADC initialization successful; LM35 signal detected.	OK
	2. Apply a change in temperature near the LM35 sensor (e.g., warm air from hand).	The ADC must register a varying analog voltage according to the LM35 scale (10 mV/°C).	Voltage variation was correctly detected by the ADC when temperature changed.	OK
	3. Observe PWM duty cycle response.	The LED brightness should increase or decrease based on the temperature thresholds defined in the system.	LED brightness adjusted smoothly according to temperature changes.	OK
	4. Monitor the digital value processed internally.	ADC digital output must follow the expected conversion curve without abrupt jumps or invalid readings.	ADC output values were stable and within expected ranges.	OK
	5. Validate diagnostic interpretation.	The system should correctly interpret ADC readings according to the defined logic (normal / high temp / over-temp).	Diagnostic logic correctly classified the temperature ranges.	OK
	6. Confirm end-to-end behavior.	ADC → logic → PWM chain must operate consistently with no delays, glitches or unexpected LED behavior.	System responded consistently across the full chain; no glitches observed.	OK
Comments				

Figure 12. Unit test 2

This test evaluates the performance of the ATmega328P's analog-to-digital converter using the thermistor temperature sensor as the input signal. When the temperature around the sensor changes, the system automatically adjusts the PWM duty cycle, producing a visible variation in LED brightness. The test confirms that the ADC correctly converts the analog voltage and that the associated logic interprets the digitalized value according to the thresholds defined in the diagnostic system

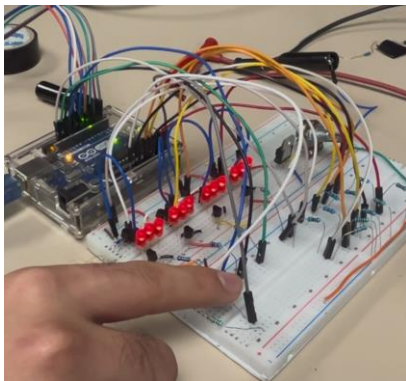


Figure 13. Test 2 before the temperature change

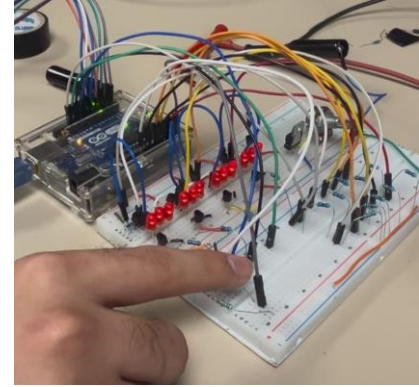


Figure 14. Test after the temperature change

3. Integration test

Integration tests validate the behavior of the system when multiple peripherals and software modules operate together. Unlike unit tests—which evaluate each peripheral in isolation—these tests confirm that the complete diagnostic chain functions coherently as a whole. For the LED diagnostic prototype, integration tests ensure that electrical and thermal anomalies acquired through the ADC are correctly interpreted by the diagnostic logic, stored in EEPROM, propagated through PWM and GPIO outputs, and reported to the user via UART.

	IT 1 / 03-DIC-2025				
Team	OPMobility – Diagnóstico LED				
Responsible					
Description	Validation of the complete diagnostic chain when an LED channel is physically disconnected (open-circuit fault).				
Function	End-to-end detection, protection, logging, and diagnostic reporting of an electrical failure.				
Components	ATmega328P (ADC, PWM/GPIO, UART), LED board, USB cable, PC with serial monitor.				
Hardware	ATmega328P @ 16 Mhz, 10-bit ADC, PWM outputs, EEPROM for fault counter, UART @ 115200 bps.				
Procedure	Initial conditions	Expected behavior	Actual behavior	Result	
	1. Physically disconnect one LED channel from the LED board.	System powered on; UART active; LED channels operational.	ADC detects abnormal voltage corresponding to an open-circuit condition.	The ADC detected a voltage outside the valid operating range for a normal LED channel.	OK
	2. Observe the diagnostic event.	Channel remains disconnected.	Fault-handling logic marks the channel as "damaged."	The MCU correctly classified the channel as damaged.	OK
	3. Observe LED output behavior.	Channel flagged as faulty.	PWM/GPIO outputs deactivate the affected LED channel.	The LED channel was deactivated immediately.	OK
	4. Verify EEPROM error.	Fault registered in logic.	EEPROM increments the error counter by one.	EEPROM stored the updated error count correctly.	OK
	5. Monitor UART output.	UART @ 115200 bps, serial monitor open.	Diagnostic message reports the open-circuit fault and updated error value.	UART transmitted the correct diagnostic message with proper formatting.	OK
	6. Evaluate full diagnostic chain coherence.	System running normally except for the disconnected channel.	ADC → Logic → PWM → EEPROM → UART chain operates consistently without data corruption.	All subsystems responded coherently, with no delays, errors, or corrupted data.	OK
Comments	We verified most of the actual behavior in the previous tests, so we can consider the results for this test as satisfactory.				

Figure 15. Integration test

This integration test validates the correct operation of the complete diagnostic chain when an LED channel is physically disconnected. The ADC detects the abnormal voltage associated with an open circuit, the fault-handling logic marks the channel as damaged, PWM/GPIO outputs deactivate the affected LED, the event is stored in EEPROM through the updated error counter, and diagnostic messages are transmitted over UART. This test

confirms the system’s ability to respond coherently and reliably to real electrical failure conditions.

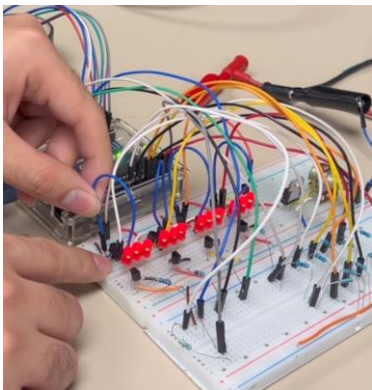


Figure 16. Integration test before generating the first short circuit

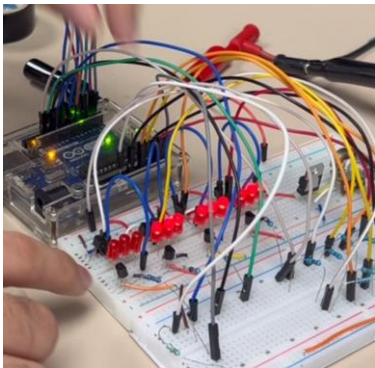


Figure 17. Integration test with one error

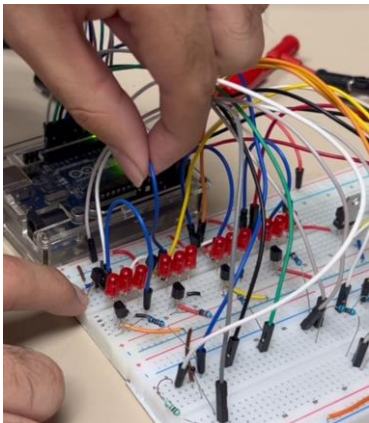


Figure 18. Integration test with two error, off all the system

D. Infrastructure

1.- Requirements

ID	Requirement (Functional)	Description / Justification
FR-01	The system shall acquire six analog signals through the ADC.	Required to measure LED channels, temperature (LM35) and VCC for diagnostic purposes.
FR-02	The system shall detect LED open-circuit and LED short-circuit conditions based on ADC thresholds.	Core diagnostic function for the LED channels.
FR-03	The system shall detect overvoltage and undervoltage conditions in the supply line.	Ensures the system responds to abnormal power states.
FR-04	The system shall disable a LED channel when a fault is detected.	Safety and protection function.
FR-05	The system shall report diagnostic messages through UART at 115200 bps.	Required for human-machine interaction and debugging.
FR-06	The system shall allow toggling the LED channels using the keypad 4*4.	Provides direct user control through the keypad 4*4.
FR-07	The system shall update and store the error count in EEPROM.	Ensures fault persistence across power cycles.
FR-08	The system shall adjust LED brightness based on temperature readings.	Implements thermal derating using LM35 measurements.
FR-09	The system shall execute the fault-handling logic automatically every ADC cycle.	Ensures continuous monitoring and system autonomy.

Figure 19. Requirements table

2.- Context diagram

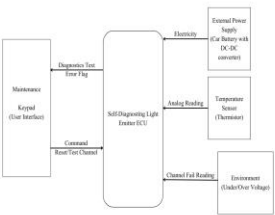


Figure 20. Context Diagram

3.- Activity diagram

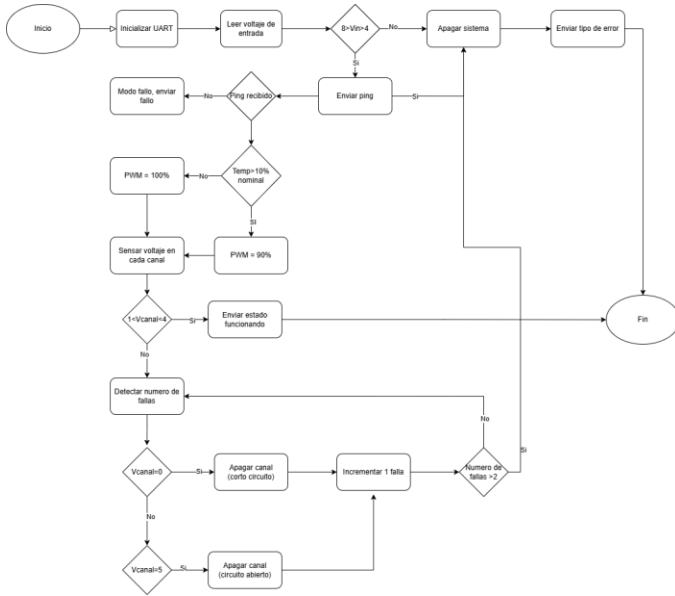


Figure 21. Activity diagram

4.- Deployment diagram

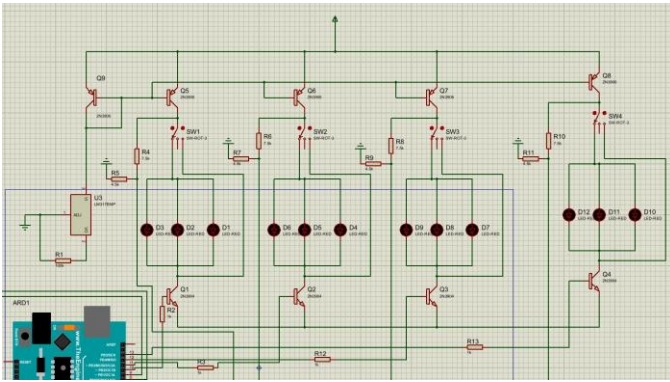


Figure 22. Deployment diagram

E. Resources used

1.- Hardware

The hardware used in this prototype was selected to meet the functional and non-functional requirements defined by the partner organization. Each component contributes directly to the diagnostic capabilities of the system, enabling voltage monitoring, thermal protection, fault detection, and user interaction. The following paragraph provides an integrated justification of the peripherals, sensors, actuators, and support electronics used in the implementation of the LED diagnostic prototype.

At the core of the system, the ATmega328P microcontroller serves as the main processing unit, offering a 10-bit ADC, UART communication interface, multiple timers (Timer0 and Timer2), EEPROM memory, and general-purpose digital I/O pins. These hardware features allow the prototype to measure analog signals from the LED board, execute fault-detection logic, control the LED channels through PWM, and store error information persistently. The microcontroller operates at 16 MHz, providing sufficient computational speed to meet the required diagnostic response time.

For sensing the electrical behavior of each lighting channel, the system uses a voltage-sensing network consisting of resistor dividers connected to the ADC inputs. This network conditions the LED line voltages into safe ranges for the microcontroller, enabling detection of open-circuit, short-circuit, undervoltage, and overvoltage conditions. To monitor the thermal state of the system, and thermistor precision temperature sensor is included. Its linear output of 10 mV/°C allows the prototype to adjust LED brightness dynamically under high-temperature scenarios, contributing to the system's thermal protection requirements.

For user interaction, a 4×4 matrix keypad is integrated through GPIO pins configured in a matrix scan arrangement. This keypad allows enabling or disabling individual LED channels, requesting system status and reading error counts, making it an essential human-machine interface for prototype validation. The lighting outputs are implemented using a 4-channel LED board, which acts as the load under test for the diagnostic logic. Each channel is controlled through PWM signals derived from Timer0 interrupts, allowing the system to apply real-time modulation based on operating conditions.

To ensure safe and stable operation, the system is powered by a regulated 5 V supply, suitable for both the microcontroller and the sensing components. The onboard EEPROM of the ATmega328P is used without external memory, as it provides enough storage capacity to maintain persistent error counters as required by the specification.

2.- Software

The development of the diagnostic prototype required several software tools used for programming, debugging,

simulation and schematic design. All firmware was developed in C using AVR-GCC and compiled through the AVR toolchain, ensuring direct register-level control of the ATmega328P peripherals (ADC, UART, timers, GPIO and EEPROM). The programming environment used was Atmel Studio or Arduino IDE (depending on the workflow), allowing integrated code compilation, serial monitoring and device flashing.

For the hardware design and verification, the system schematic and layout were created using Proteus Design Suite, which enabled simulation of the LED channels, the keypad, analog sensing networks and the microcontroller peripherals before physical implementation. Analog behavior—such as voltage dividers and temperature sensor output—was analyzed using Proteus simulation tools to validate the sensing thresholds and ensure correct scaling for the ADC inputs.

Serial communication testing and debugging were performed using terminal applications such as the Arduino Serial Monitor, PuTTY or TeraTerm, which allowed real-time observation of diagnostic messages, fault logs and system states. These tools facilitated validation of the UART interface at 115200 bps and verification of the correct formatting and timing of transmitted data.

Overall, the selected hardware and software resources provide full support for the development, validation and integration of the LED diagnostic system, ensuring compliance with the functional and quality requirements established by the partner organization.

III. RESULTS

The evaluation of the prototype confirmed that the system successfully met all the requirements established for the challenge. The controller was capable of detecting overvoltage and undervoltage conditions, logging the number of faults that occurred, and executing the defined safety actions—such as shutting down LED channels when two simultaneous faults were detected. Additionally, the integration of the thermistor enabled dynamic adjustment of LED intensity based on temperature, reducing the risk of overheating.

Functionally, the system operated in a stable manner: the serial communication responded correctly, the channels reacted to diagnostic events, and the fault

counter remained within expected behavior. Although some minor errors were not perceptible to the human eye, they did not affect the overall performance of the prototype or the validation of the project objectives.

These results confirm that the proposed architecture, together with the sensing, control, and diagnostic techniques implemented, achieves a functional and reliable representation of a basic automotive lighting module.

IV. CONCLUSIONS

A. Summary of Findings and Alignment with Objectives

The project enabled the development of a system capable of monitoring critical operating variables (voltage, current, and temperature) and responding to abnormal conditions through controlled actions. The objectives were fully achieved: fault detection, fault logging, safe channel control, and functional bidirectional communication. The prototype demonstrates the feasibility of implementing basic diagnostic functions using a low-cost microcontroller while maintaining stable and reproducible behavior.

B. System Limitations

Although functional, the platform presents limitations inherent to its prototypical nature. The use of an Arduino UNO restricts scalability toward real automotive applications, and some faults are not fully perceptible visually, which complicates evaluation without additional instrumentation. Furthermore, the accuracy of thermal and electrical sensing could be improved by incorporating components designed for more demanding environments.

C. Reflection on Collaboration

The organization of the team was based on a strategic division of work to avoid oversaturation in any single area. Three main domains were established—hardware, software, and documentation—yet all team members demonstrated the ability to operate and explain the complete functionality of the system. One member particularly excelled in software development, which accelerated this critical stage, while contributions in the remaining areas were balanced.

The team faced significant unforeseen challenges, such as the failure of the PIC microcontroller, which delayed progress and required a reevaluation of the technical approach. Additionally, coordinating schedules proved difficult since two members were from out of town and two were local, limiting the available overlap for joint work sessions. Despite this, collaboration remained effective through constant communication, clear prioritization of tasks, and mutual support. In many cases, one member directly requested what was needed, and another continued work remotely, reducing idle time and avoiding bottlenecks.

This experience highlighted the importance of flexibility, shared responsibility, and thoughtful distribution of talent within a technical project.

D. Ethical Reflection and Project Integrity

From the initial system design, ethical considerations related to user safety and device reliability were taken into account. Priority was given to the immediate shutdown of the module when voltage variations could compromise the thermal integrity of the system, thereby preventing risks such as overheating or damage to the LEDs.

Ethical decisions were also made regarding the quality and accuracy of the work. The team chose to rely on real measurements rather than simulated data, even when this meant dealing with variations or uncertainties inherent to hardware. Limitations were documented instead of concealed, and exaggerated or non-reproducible results were intentionally avoided. This approach reinforces the values of integrity in technological development and in experimental validation.

E. Suggestions for Improvement and Future Work — Translation

For a more robust version of the system, it is proposed to design a professional PCB that enables the integration of automotive-grade components and enhances overall reliability. It is also recommended to migrate from the Arduino UNO to an automotive-

oriented microcontroller, or alternatively, to a more economical, efficient, and scalable option. Implementing real automotive LEDs is suggested to achieve a more representative validation of in-field behavior. Another improvement involves adding a visual interface or menu (HUD) that allows a technician or repair specialist to intuitively review system faults and operational states. Finally, incorporating higher-precision sensors and improving electronic protection strategies is recommended to align the design more closely with industry-level automotive standards.

V. REFERENCES

- [1] OPmobility. (2025, September 11). *OPmobility leader in sustainable and connected mobility*.
<https://www.opmobility.com/en/>
- [2] Attal (2025) *An Approach for Automotive ECU Diagnosis via Ethernet Snooping & Microcontroller Tracing*
https://www.researchgate.net/publication/395226474_An_Approach_for_Automotive_ECU_Diagnosis_via_Ethernet_Snooping_Microcontroller_Tracing
- [3] Wolf, M. (2022). Automotive and aerospace systems. In Elsevier eBooks (pp. 437–452).
<https://doi.org/10.1016/b978-0-323-85128-2.00009-8>
- [4] Key, S. (2023, April 27). Why Is Prototyping Important? 9 Powerful Benefits Of Prototypes | inventRight. *inventRight*.
<https://inventright.com/innovate-this/why-is-prototyping-important/>
- [5] GeeksforGeeks. (2024, April 30). *8 reasons Why do we need IDE*. GeeksforGeeks.
<https://www.geeksforgeeks.org/python/why-do-we-need-an-ide/>
- [6] GeeksforGeeks. (2025, July 23). *What is an IDE? Integrated Development Environment*. GeeksforGeeks.
<https://www.geeksforgeeks.org/blogs/what-is-ide/>
- [7] GeeksforGeeks. (2025a, July 23). *Microcontroller and its Types*. GeeksforGeeks.

<https://www.geeksforgeeks.org/digital-logic/microcontroller-and-its-types/>

[8] GeeksforGeeks. (2025c, August 29). Von Neumann Architecture. GeeksforGeeks.

<https://www.geeksforgeeks.org/computer-organization-architecture/computer-organization-von-neumann-architecture/>

[9] GeeksforGeeks. (2025d, September 1). Harvard Architecture. GeeksforGeeks.

<https://www.geeksforgeeks.org/computer-organization-architecture/harvard-architecture/>

[10] GeeksforGeeks. (2024b, May 28). What is Prototyping? Definition, Types, Qualities and more. GeeksforGeeks.

<https://www.geeksforgeeks.org/software-engineering/what-is-a-prototype-and-how-to-create-it/>

[11] Lee, S. (n.d.). Interfacing Microcontrollers with Other Devices.

<https://www.numberanalytics.com/blog/interfacing-microcontrollers-with-other-devices>

[12] Seritan, G., Raicu, C., & Enache, B. (n.d.). Single-Wire control and fault detection for automotive exterior

lighting systems. *Sensors*, 23(14), 6521.

<https://doi.org/10.3390/s23146521>

[13] Martínez-Pérez, J. R., Carvajal, M. A., Santaella, J. J., López-Ruiz, N., Escobedo, P., & Martínez-Olmos, A. (2024). Advanced Detection of Failed LEDs in a Short Circuit for Automotive Lighting Applications. *Sensors*, 24(9), 2802. <https://doi.org/10.3390/s24092802>

[14] Li, S., Frey, M., & Gauterin, F. (2024). An Innovative Technique for Fault Analysis of Electric Automated Vehicles. *Vehicles*, 6(4), 1995-2010.

<https://doi.org/10.3390/vehicles6040098>

[15] Sean Kennedy. (2021, July 26). STM32CubeIDE programming, debugging, and Running [Video]. YouTube.

<https://www.youtube.com/watch?v=BVC7KaUNCS8>

[16] Bus Systems – CAN/FD, FlexRay, Ethernet, LIN, MOST, K-Line in use | Softing. (n.d.).

<https://automotive.softing.com/standards/bus-systems.html>

[17] LIN vs CAN vs FlexRay vs MOST: A Detailed Comparison. (n.d.). RF Wireless World.

<https://www.rfwireless-world.com/terminology/lin-vs-can-vs-flexray-vs-most>